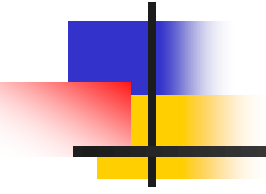


Arba Minch University

Faculty of Computing and Software Engineering

- Course Code ------- SENG 1061
- Course Title -------- **Fundamental Software Engineering**
- Degree Program --- SE 2nd Year Sem II

Fundamental of Software Engineering (SENG3164)



Chapter 01

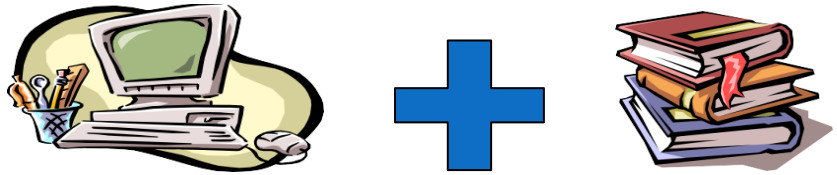
Overview of Software Engineering

Topics Covered

- Software
- Software Engineering
- Software Engineering and Computer Science
- Software Myths
- CASE
- Attributes of good software

What is Software?

- **Computer programs** and **associated documentation**



- **A Program:-** an executable code, which serves some computational purpose.
- Software engineering is an engineering discipline that is concerned with all aspects of software production.
- **Software** may be developed for a particular customer or may be developed for **products** or a general market
- **Software products** may be
  - **Generic** - developed to be sold to a range of different customer
  - **Bespoke** (custom) - developed for a single customer according to their specification

What is Software Engineering

- **Software engineering** is an engineering branch associated with development of software product using well-defined scientific
  - principles,
  - Methods
  - procedures.
- **outcome of SE:** software product which is:
  - Efficient
  - reliable



Conti…

- Software engineering is the establishment and use of sound engineering principles in order to obtain economically software that is reliable and efficiently on real machines.
- Software engineering is a **modeling activity.**
- Software engineering is a **problem-solving activity.**
- Software engineering is a **knowledge acquisition activity.**
- Software engineering is a **rationale-*driven activity*.**

Conti…

- **Modeling Activity:**
  - A model is an abstract representation of a system that enables us to answer questions about the system.
  - Software engineers deal with complexity through modeling, by focusing at any one time on only the relevant details and ignoring everything else.
  - Models are useful when dealing with systems that are too large, too small, too complicated, or too expensive
  - To experience firsthand.

Conti…

Problem solving

- Engineering is a problem-solving activity.
- In its simplest form, the engineering method includes five steps:

1. formulate the problem
2. analyze the problem
3. search for solutions
4. decide on the appropriate solution
5. specify the solution

Conti…

Knowledge acquisition

- In modeling the application and solution domain, software engineers collect data, organize it into information, and formalize it into knowledge.
- Knowledge acquisition is **nonlinear,** as a single piece of data can invalidate complete models.
- A common mistake that software engineers and managers make is to assume that the acquisition of knowledge needed to develop a system is linear

Conti…

Rationale management

- When acquiring knowledge and making decisions about the system or its application domain, software engineers also need to capture the context in which decisions were made and the rationale behind these decisions.
  - Enables software engineers to understand the implication of a proposed change when revisiting a decision.

Important progress of Software Engineering

- Ability to produce more complex software has increased
- New technologies have led to new SE approaches
- A better understanding of the activities involved in software development
- Effective methods to specify, design and implement software have been developed
- New notations and tools have been produced

Importance of software engineering

- More and more, individuals and society rely on advanced software systems.
  - We need to be able to produce reliable and trustworthy systems economically and quickly.
- It is usually cheaper, in the long run, to use software engineering methods and techniques for software systems rather than just write the programs as if it was a personal programming project.
  - For most types of system, the majority of costs are the costs of changing the software after it has gone into use.

Application types

- **Stand-alone applications**
  - These are application systems that run on a local computer, such as a PC. They include all necessary functionality and do not need to be connected to a network.
- **Interactive transaction-based applications**
  - Applications that execute on a remote computer and are accessed by users from their own PCs or terminals. These include web applications such as e-commerce applications.
- **Embedded control systems**
  - These are software control systems that control and manage hardware devices. Numerically, there are probably more embedded systems than any other type of system.

Conti…

- **Batch processing systems**
  - These are business systems that are designed to process data in large batches. They process large numbers of individual inputs to create corresponding outputs.
- **Entertainment systems**
  - These are systems that are primarily for personal use and which are intended to entertain the user.
- **Systems for modelling and simulation**
  - These are systems that are developed by scientists and engineers to model physical processes or situations, which include many, separate, interacting objects.

Conti…

- **Data collection systems**
  - These are systems that collect data from their environment using a set of sensors and send that data to other systems for processing.
- **Systems of systems**
  - These are systems that are composed of a number of other software systems.

Internet Software Engineering

- The Web is now a platform for running application/s and organizations are increasingly developing web-based systems rather than local systems.
- Cloud computing is an approach to the provision of computer services where applications run remotely on the ‘cloud’.
  - Users do not buy software buy pay according to use.

Web-based Software Engineering

- Web-based systems are complex distributed systems but the fundamental principles of software engineering discussed previously are as applicable to them as they are to any other types of system.
- The fundamental ideas of software engineering apply to web-based software in the same way that they apply to other types of software system.

Web software engineering

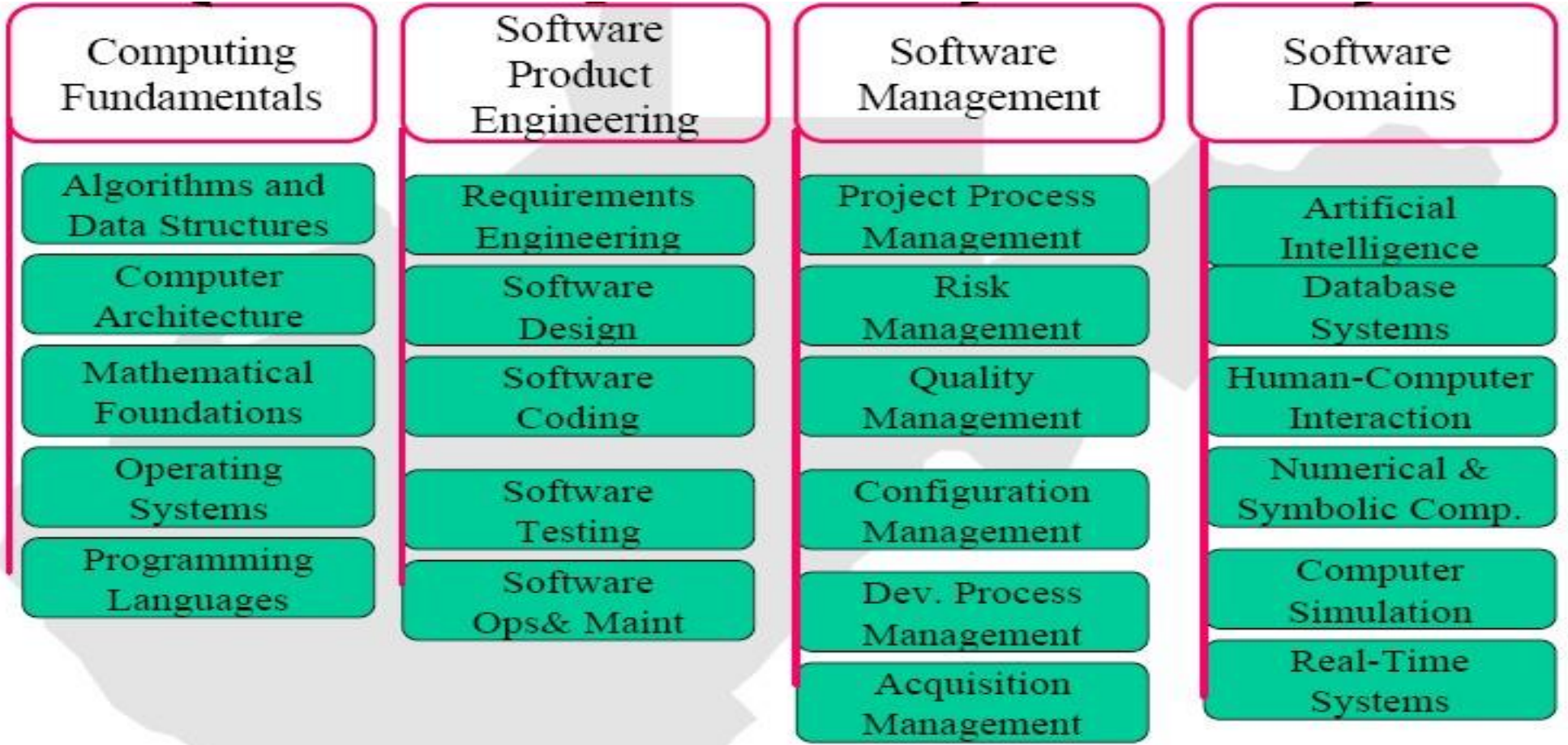
- **Service-oriented systems**
  - Software may be implemented using service-oriented software engineering, where the software components are stand-alone web services.
- **Rich interfaces**
  - Interface development technologies such as AJAX and HTML5 have emerged that support the creation of rich interfaces within a web browser.

The difference between software engineering and computer science

Computer Science	Software Engineering
is concerned with	
➢ theory ➢ fundamentals Algorithms, date structures, complexity theory, numerical methods	➢ the practicalities of developing ➢ delivering useful software SE deals with practical problems in complex software products

Computer science theories are currently insufficient to act as a complete underpinning for software engineering, BUT it is a foundation for practical aspects of software engineering

SE body of Knowledge



Source: http://www.sei.cmu.edu/pub/documents/99.reports/pdf/99tr004.pdf

Software Myths

- **Management myths**
  - *Standards and procedures for building software*
  - *Add more programmers if behind the schedule*
- **Customer myths**
  - *A general description of objectives enough to start coding*
  - *Requirements may change as the software is flexible*
- **Practitioner myths**
  - *Task accomplished when the program works*
  - *Quality assessment when the program is running*
  - *Working program the only project deliverable*

Computer-Aided Software Engineering (CASE)

- **Software systems which are intended to provide automated support for software process activities,** such as requirements analysis, system modelling, debugging and testing



Upper-CASE

- Tools to support the early process activities of requirements and design

Lower-CASE

- Tools to support later activities such as programming, debugging and testing

What are the attributes of good software?

- The software should deliver the required functionality and performance to the user and should be maintainable, dependable and usable
- **Maintainability**
  - Software must evolve to meet changing needs
- **Dependability**
  - Software must be trustworthy
- **Efficiency**
  - Software should not make wasteful use of system resources
- **Usability**
  - Software must be usable by the users for which it was designed

What are the key challenges facing software engineering?

- **Legacy systems**
  - Old, valuable systems must be maintained and updated
- **Heterogeneity**
  - Systems are distributed and include a mix of hardware and software
- **Delivery**
  - There is increasing pressure for faster delivery of software
- **Security and Trust**
- **Scale**

END

Q?

Course Syllables

- Chapter 1
- Chapter 2
- Chapter 3
- Chapter 4
- Chapter 5